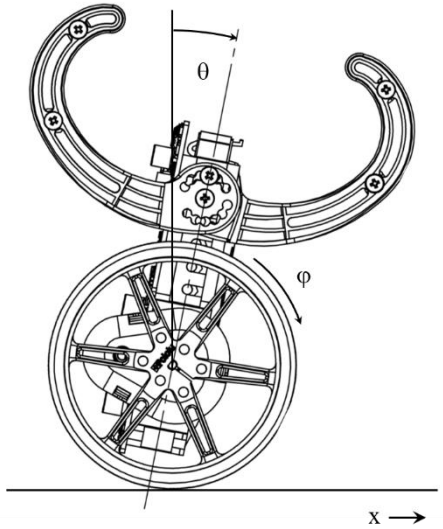
Model-Based Design Workshop

Res Jöhr

07.05.2026



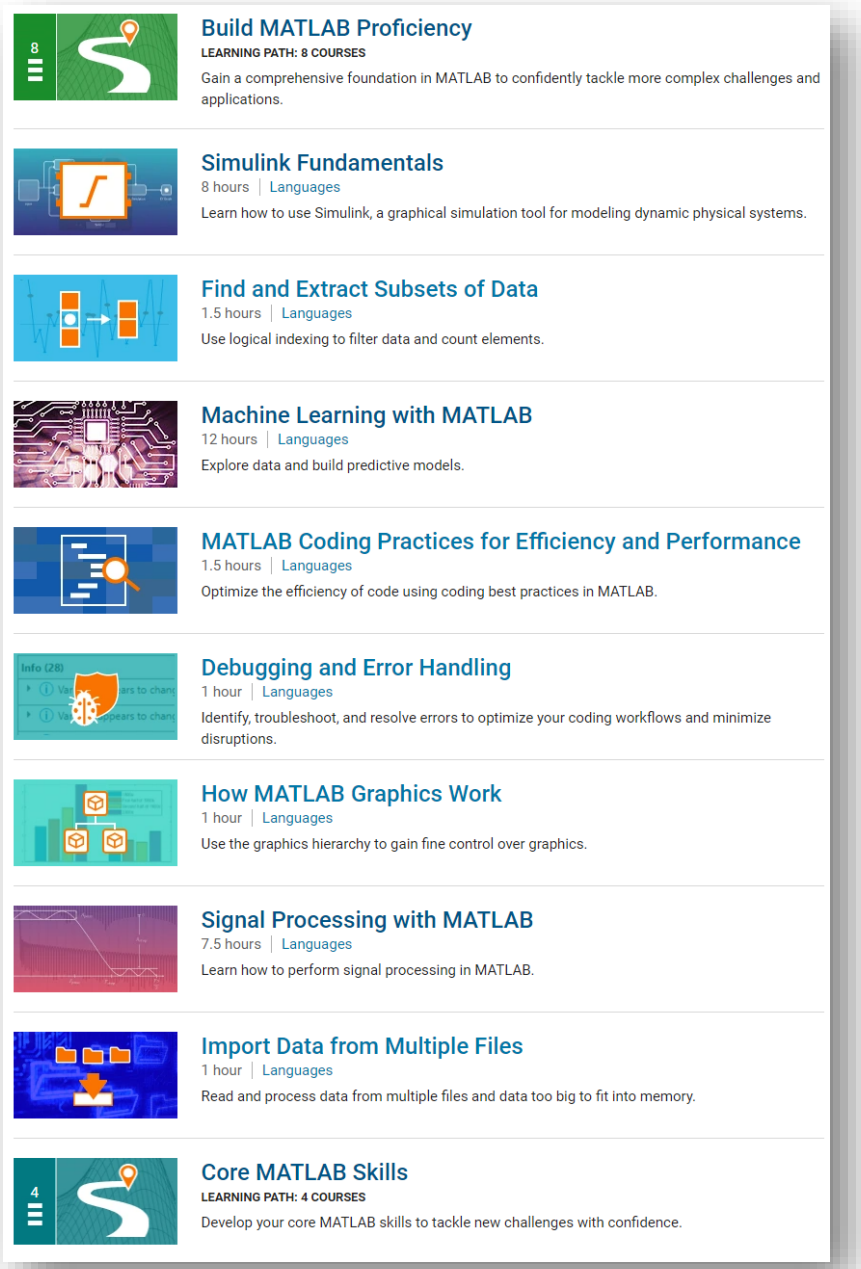
Agenda

Time	Topic
12:30-12:40	Introduction
12:40-13:00	Simulink / Simscape refresher
13:00-13:30	Motor-Control (Hands-On)
13:30-14:00	Mechanical Modeling (Hands-On)
14:00-14:15	Walk through additional topics
14:15-14:30	Wrap-up and open discussion

Your expectations

Online Trainings

- Access to *all* MATLAB and Simulink online training courses
  - 83 Self-paced courses (24 Onramps)
  - 15 Learning Paths
- Interactive lessons with immediate feedback
- Choose to take individual lessons, short courses, or full learning paths
- Wide range of available topics—new courses and topics added every few months
- Progress report and completion certificate



Sample list – see all courses and learning paths currently available at: http://matlabacademy.mathworks.com

Accelerate your MATLAB and Simulink productivity with Generative AI

1. MATLAB and Simulink Copilots

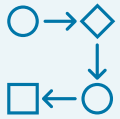
optimized for MATLAB and Simulink

2. External GenAI Agents

use MATLAB or Simulink with external Agents via MCP or VS Code

3. Develop GenAI-powered Applications

call LLMs from MATLAB code



4. Other GenAI capabilities

ChatGPT, MathWorks documentation, etc.

Logistics

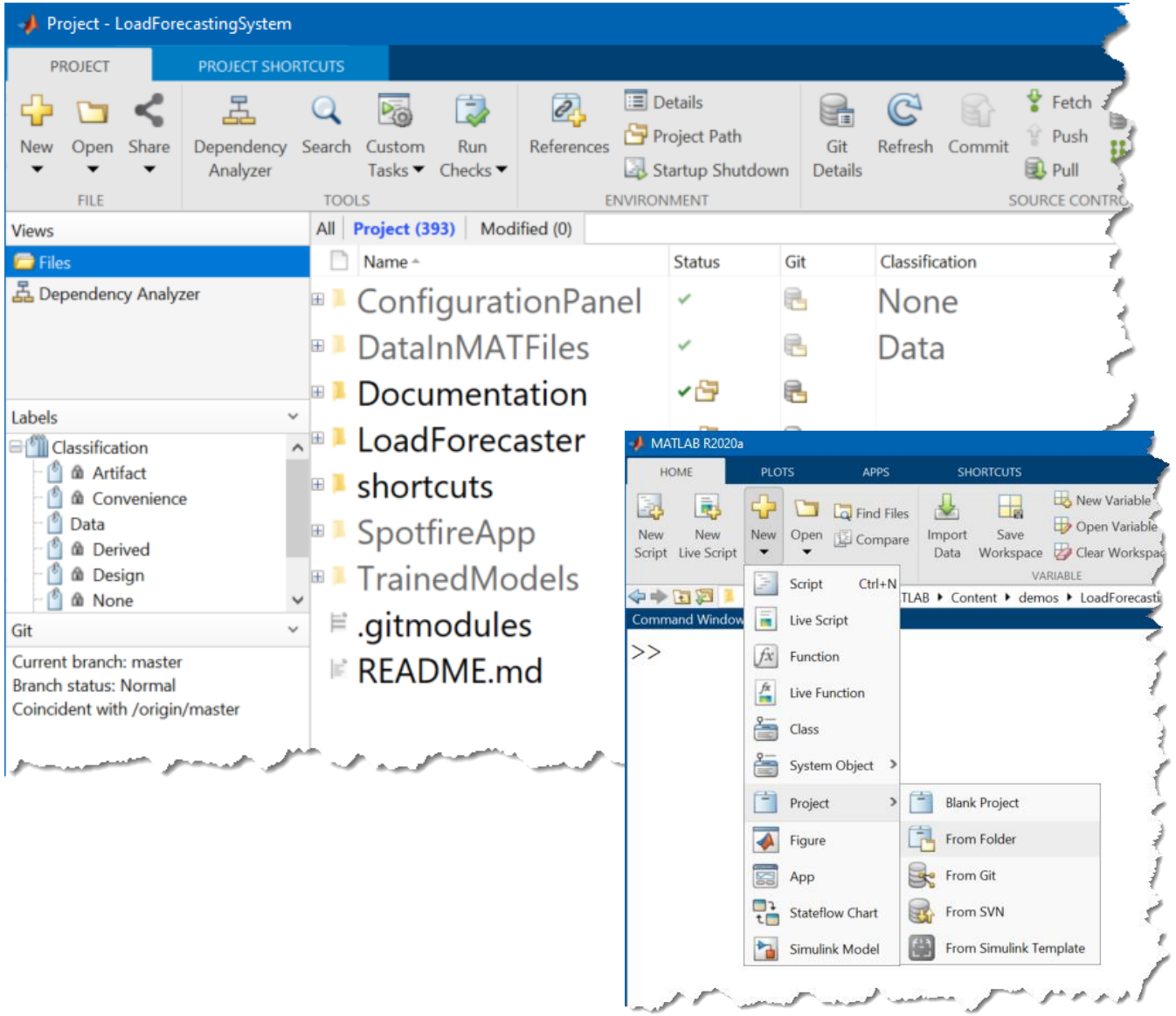
- Required: MathWorks Account associated with your EPFL mail
- Hands-On parts are done in the Online Version of MATLAB and Simulink
- Get the materials here:

  https://github.com/resjoehr/Balancing_Robot_MBD_Workshop

Click here

Projects

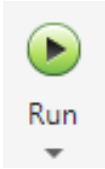
- File Organization and Distribution System
- Manage:
  - Path
  - Startup and Shutdown
  - Project-level Shortcuts
  - Tasks and Checks
- Integration with Source Control
- Try an example
- Onramp Module



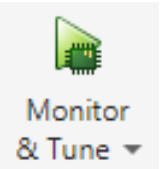
Simulink for Maker Projects

Types of execution:

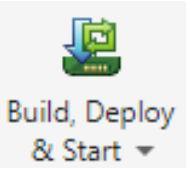
- **Simulation mode**: Runs on desktop/cloud (no hardware connectivity)



- **External mode**: Runs on desktop but I/O are connected to hardware. Requires blocks from support package.



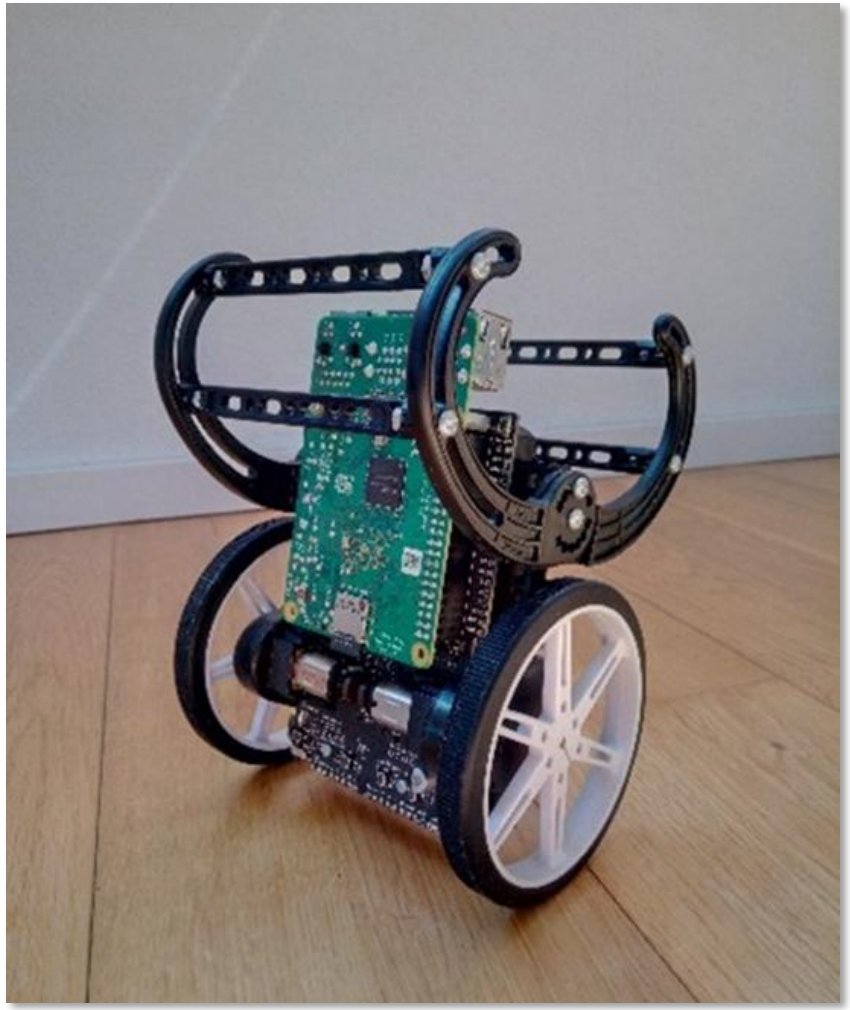
- **Deployed mode**: Generated code is running stand-alone on the target.



Note: Today, we focus on model building and simulation. Hardware parts will be demonstrated, or experimental data will be provided.

Hardware Setup: Pololu Balboa Balancing Robot

- Arduino board
  - Motors
  - Encoders
  - IMU Sensor
- Raspberry Pi 3B+
  - Connected to Arduino via I2C
  - Websocket for control over WiFi



- Robot supplier page: https://www.pololu.com/product/3575

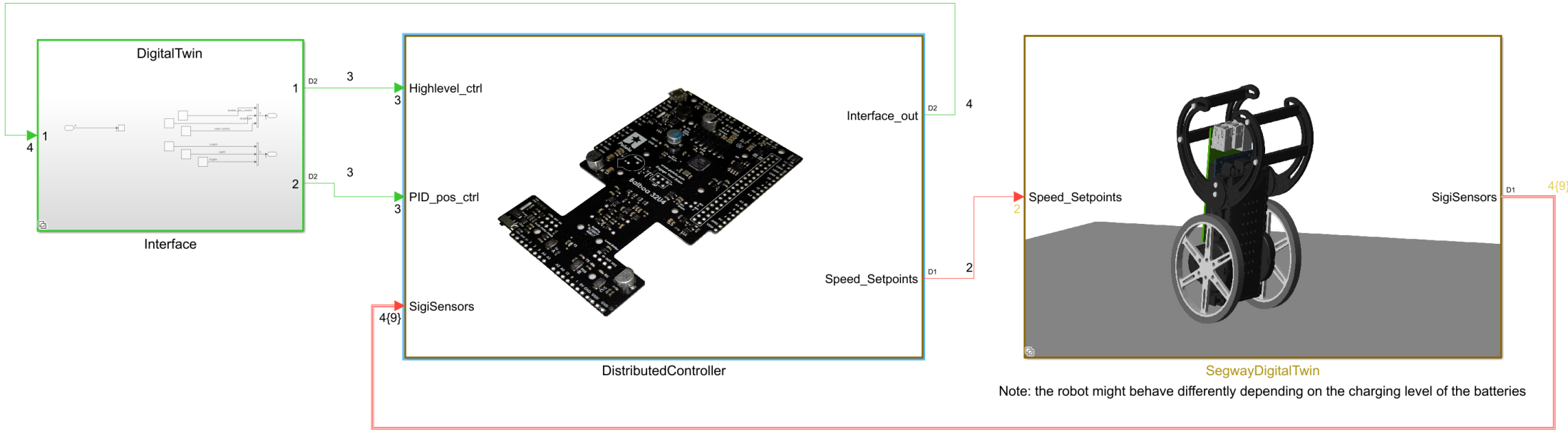
Aim

- Use digital twin in Simulink to design control algorithms
- Generate C/C++ code and deploy to hardware (reduce dev time)

Why should we use models?

- Single source of truth
- No need to have hardware for testing
- Test algorithms without risk to damage hardware
- Easier to get insights in case of failure (signal loging)

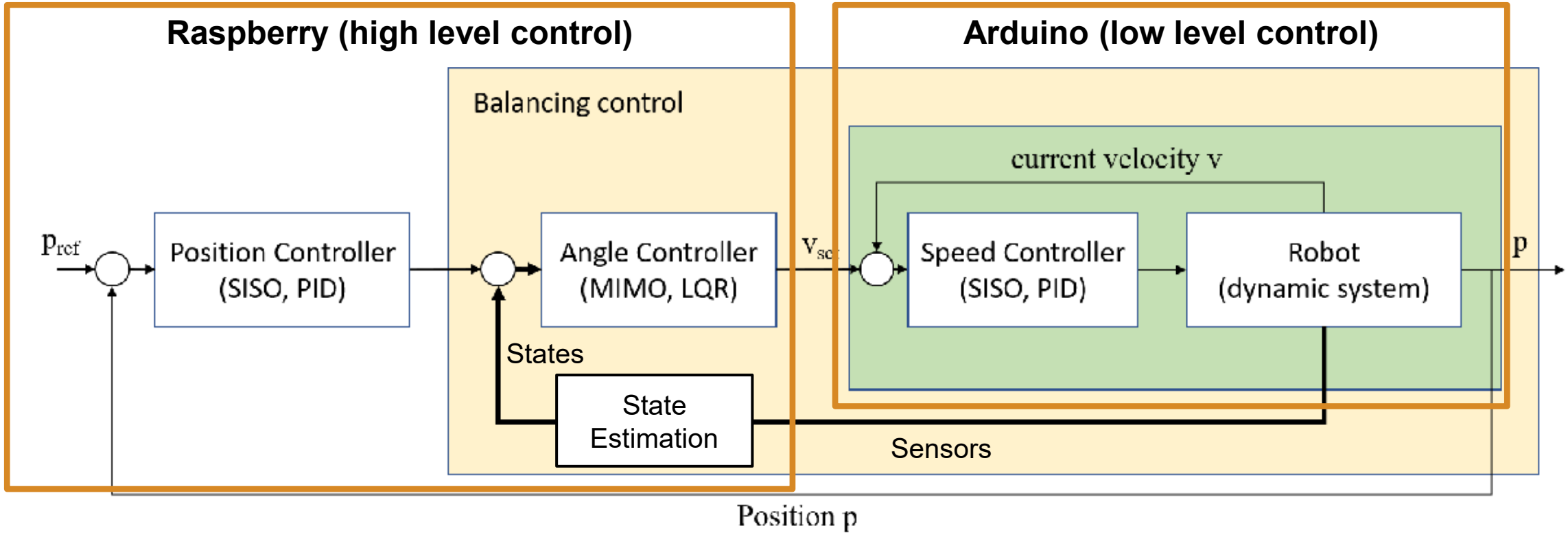
Aim: Develop Controllers using in-silico Model



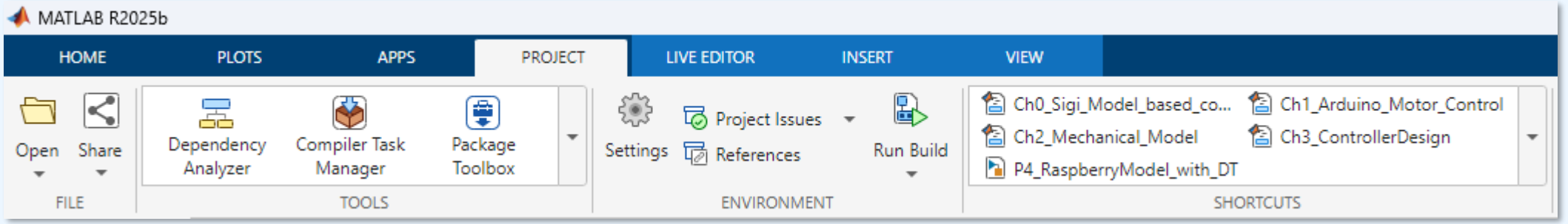
1. Break down model into physical and software components
2. Model the hardware part.
3. Use the plant model for the development of the algorithms.
4. Drop plant model and deploy controller to real target.

Where should we start?

- Consider desired control architecture
  - Position Control -> Balancing Control -> Motor Speed Control
- Align with hardware functionality and constraints
- Build the model using bottom-up approach



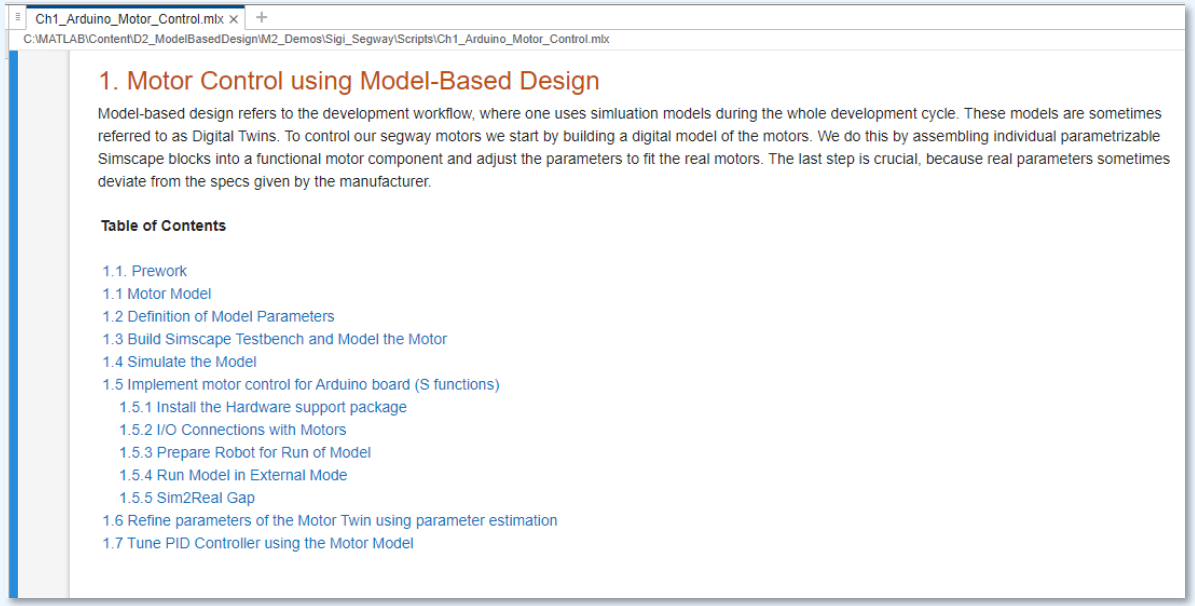
Demo Part: Motor Modelling



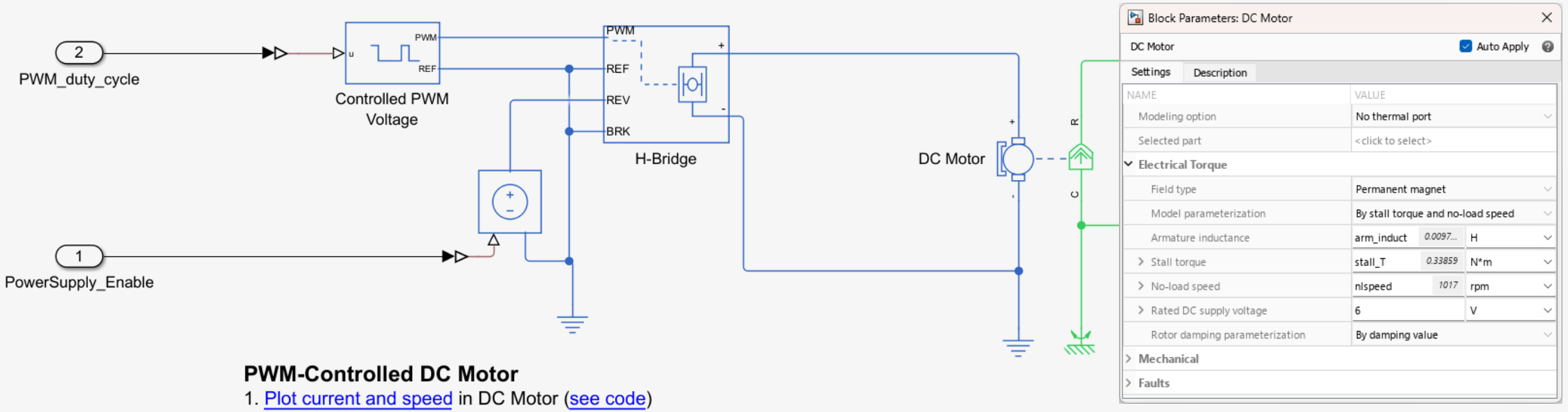
- Open script Ch1_Arduino_Motor_Control.mlx (Project Shortcut)

Task:

- Implement DC motor
- Controlled via H-Bridge
- Driven by PWM signal
- Motor gears
- Wheel connection for Simscape MB
- Save as reusable library block



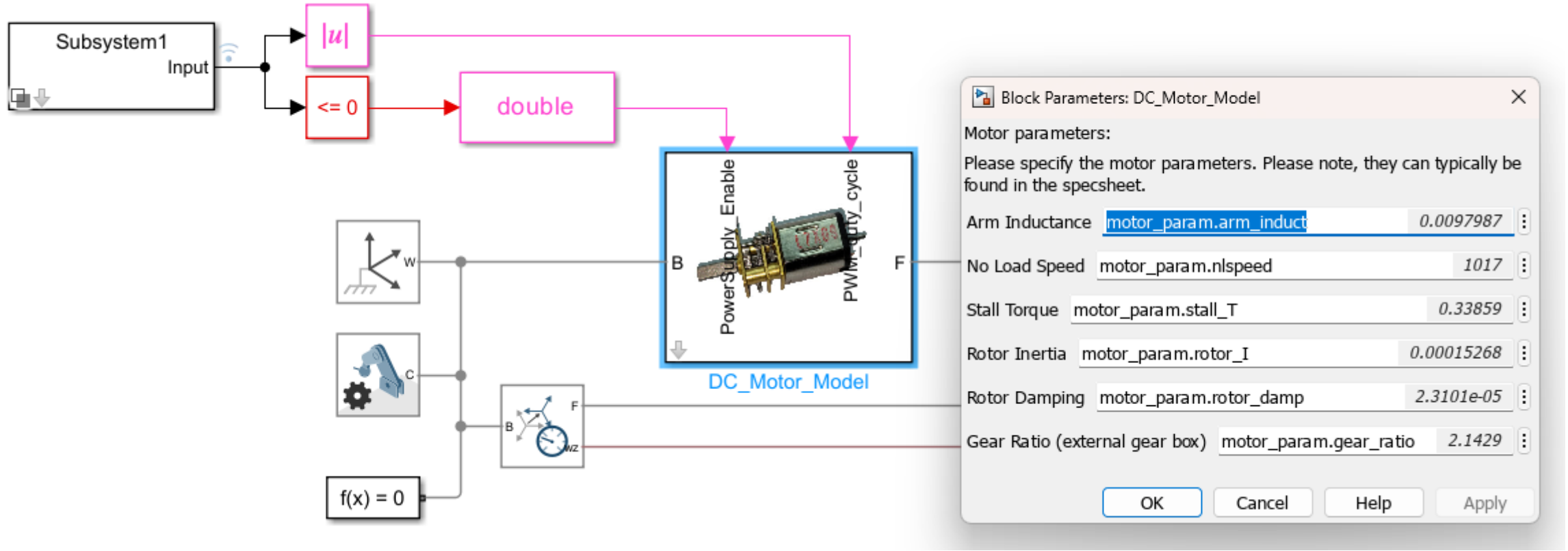
Motor Modelling using Simscape Electrical



- Build the model using electrical components
- Adjust motor model with the specs from datasheet

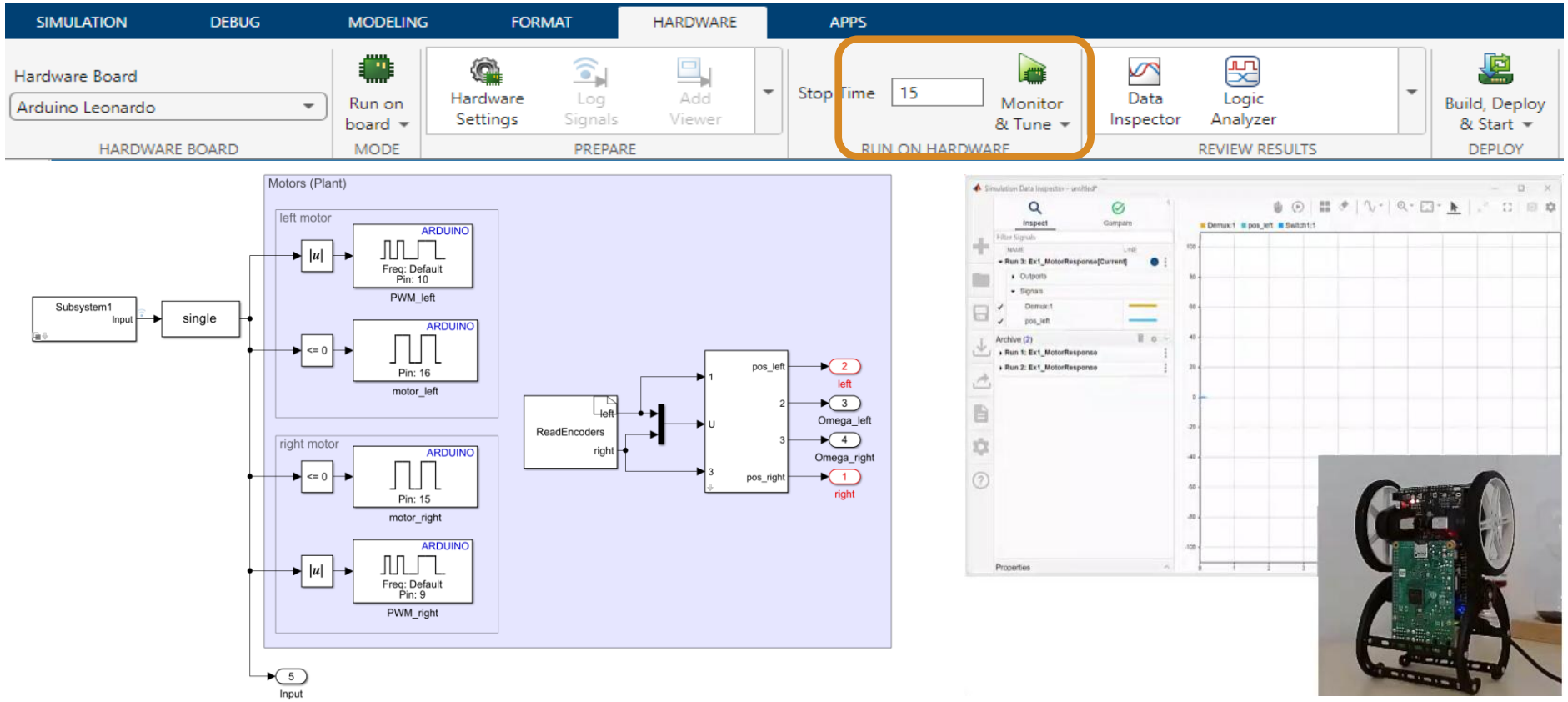
- Simscape Electrical Onramp: https://matlabacademy.mathworks.com/details/circuit-simulation-onramp/circuits
- Example from Documentation: https://www.mathworks.com/help/sps/ug/example-modeling-a-dc-motor.html

Parametrizable Model Libraries



- Documentation: Create Custom Library: https://www.mathworks.com/help/simulink/ug/creating-block-libraries.html
- Learn More: Video “Libraries in Simulink made Easy” https://www.youtube.com/watch?v=xWmIACn5Te0

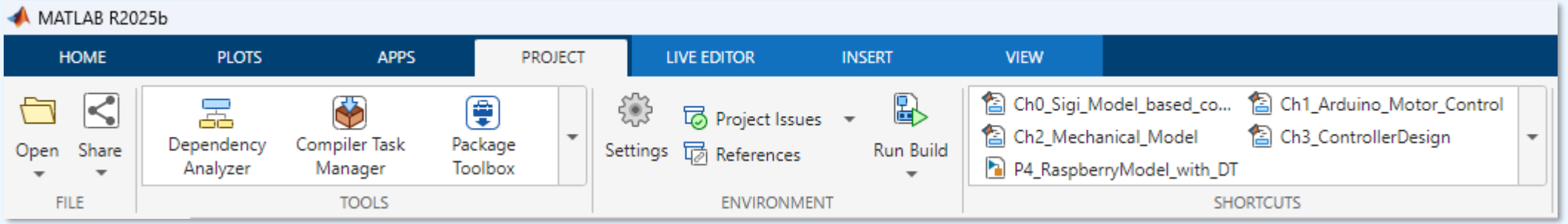
Parameter Optimization using Hardware Connection



- Control real motor from within Simulink using External Mode
- Integrate 3rd party code for the encoders into S-function

- Simulink Support Package for Arduino: https://www.mathworks.com/matlabcentral/fileexchange/40312-simulink-support-package-for-arduino-hardware

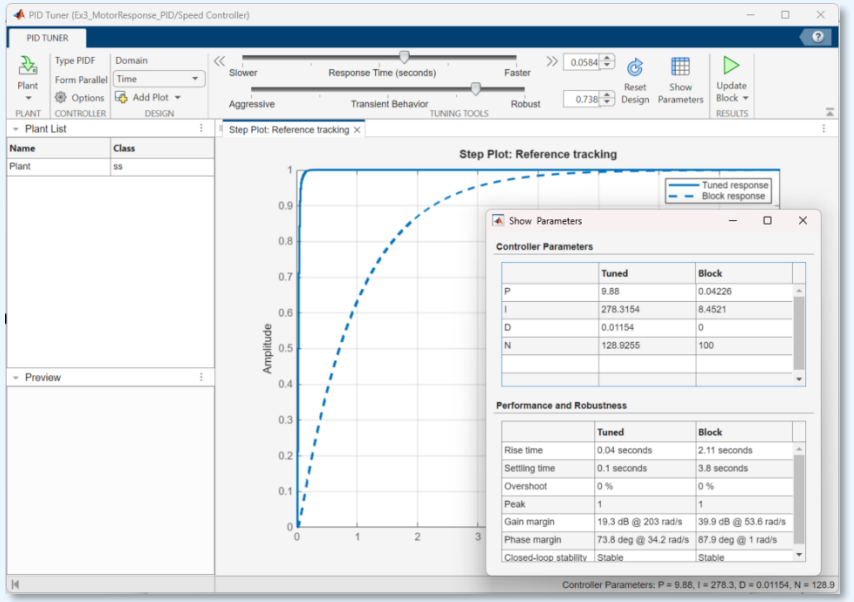
Hands-On: Parameter Optimization



- Open script Ch1_Arduino_Motor_Control.mlx (Project Shortcut)

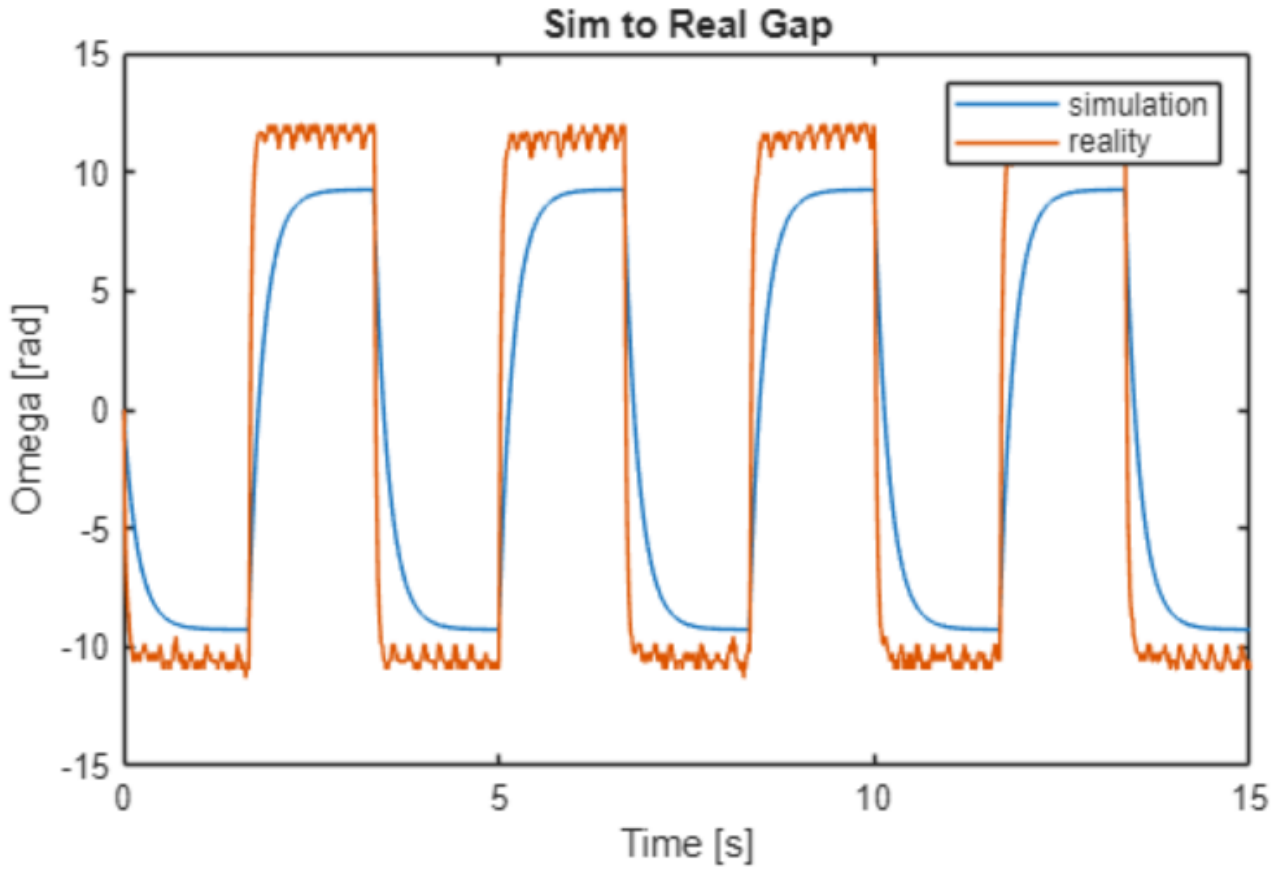
Task:

- Compare model to reality
- Optimize motor parameters
- Validate new parameters
- Implement PID for speed control
- Deploy controller to Arduino

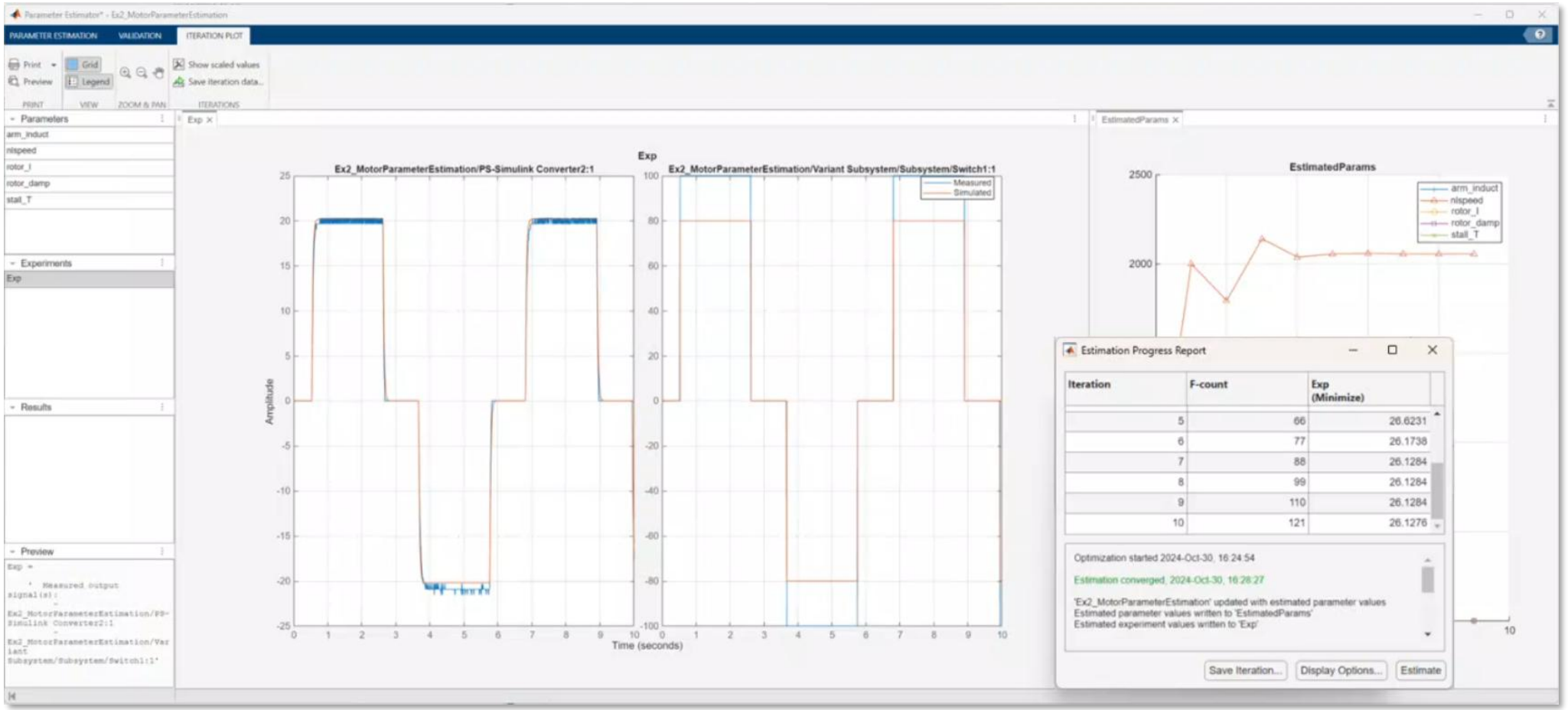


Sim to Real Gap

- Initial models are often different from the real world!



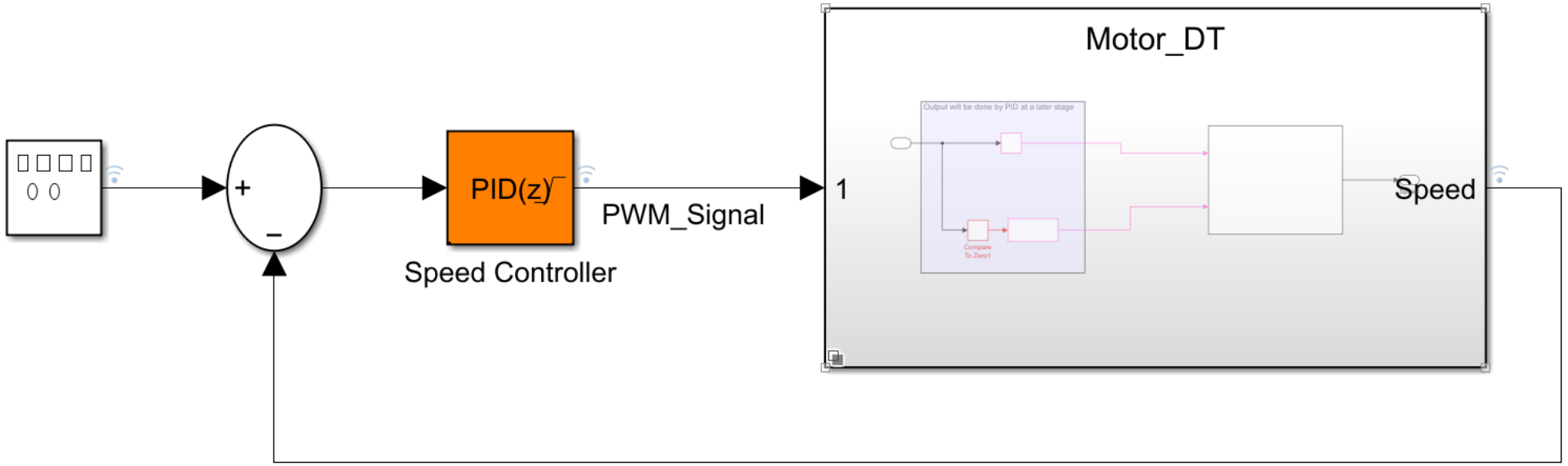
Parameter Estimation



- Optimize parameters of motor to minimize Sim to Real Gap

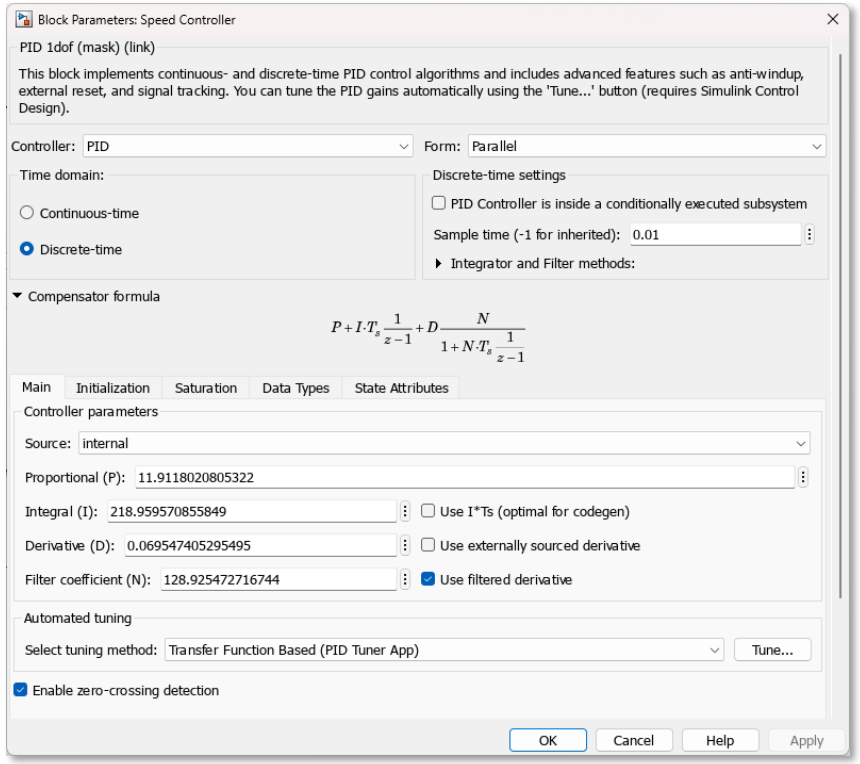
- Example about parameter estimation workflow:
  https://www.mathworks.com/help/sldo/ug/dc-servo-motor-parameter-estimation.html

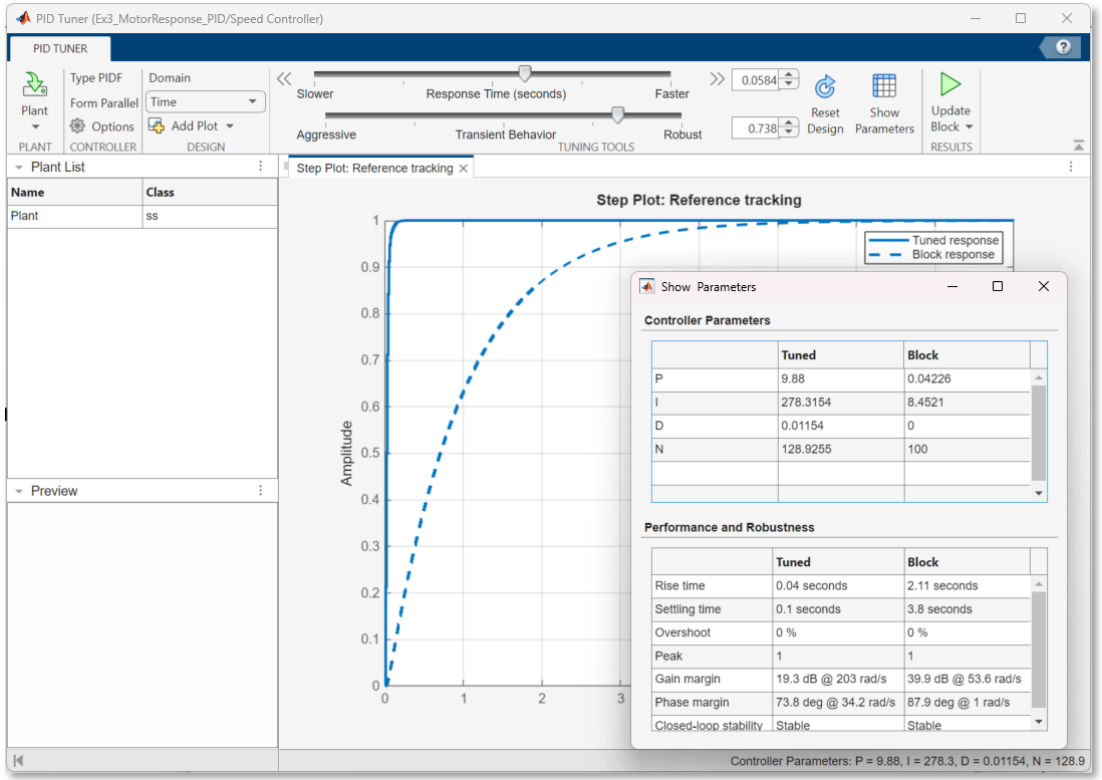
Developing a PID controller (1)



- Simulink Control Design Onramp: https://matlabacademy.mathworks.com/details/control-design-onramp-with-simulink/controls

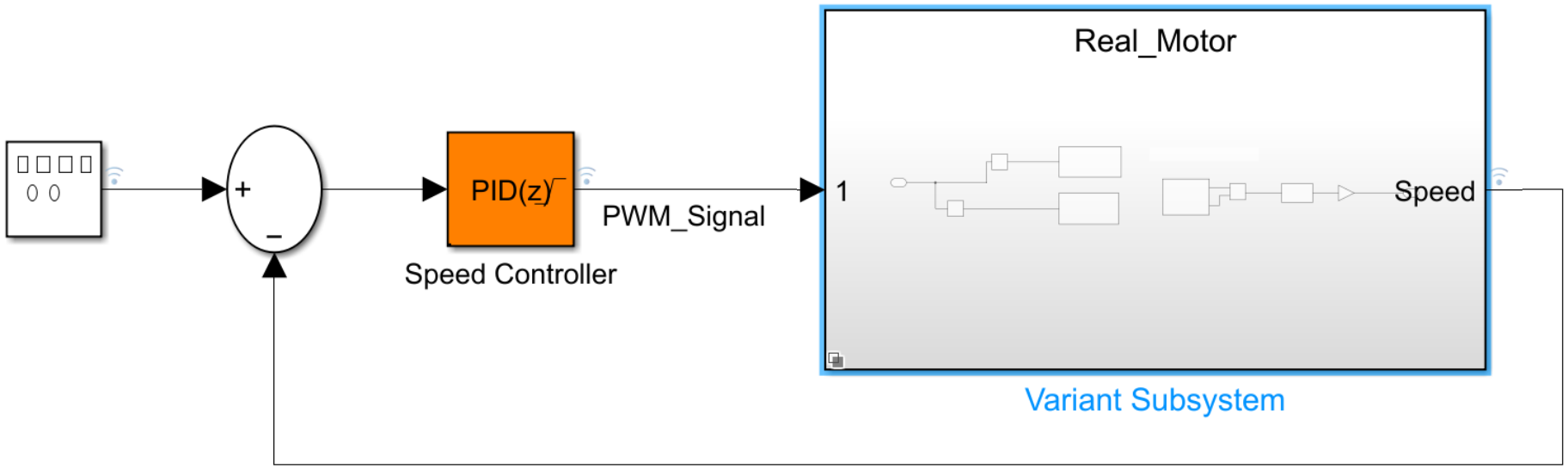
Developing PID controller (2)





Design your PID controller interactively using the PID Tuner App

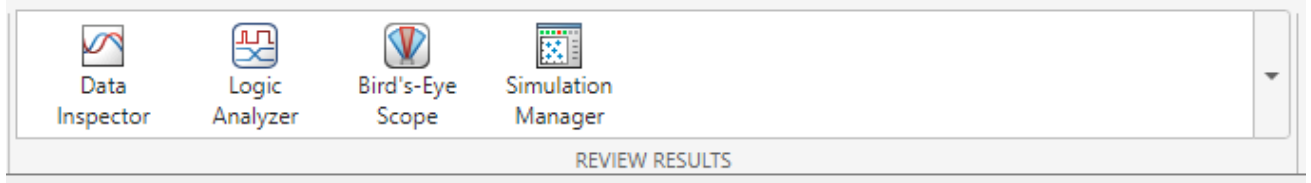
Developing PID controller (3)



- Use variant model to switch between different implementation

- Documentation Variant Subsystems: https://www.mathworks.com/help/simulink/var/what-is-a-variant.html

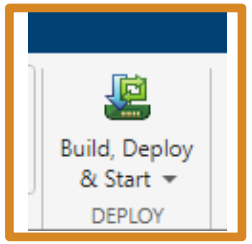
Simulink Data Inspector



- Compare logged results of different runs for validation

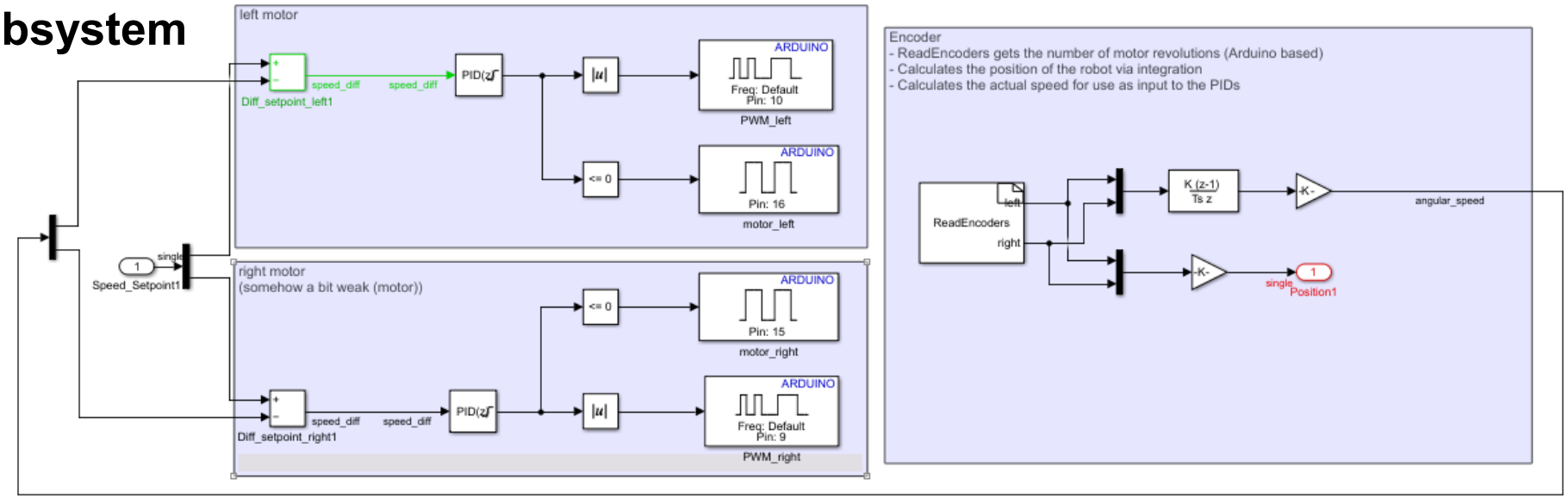
- Documentation Data Inspector: https://www.mathworks.com/help/simulink/slref/simulationdatainspector.html

Finalize the low-level motor control

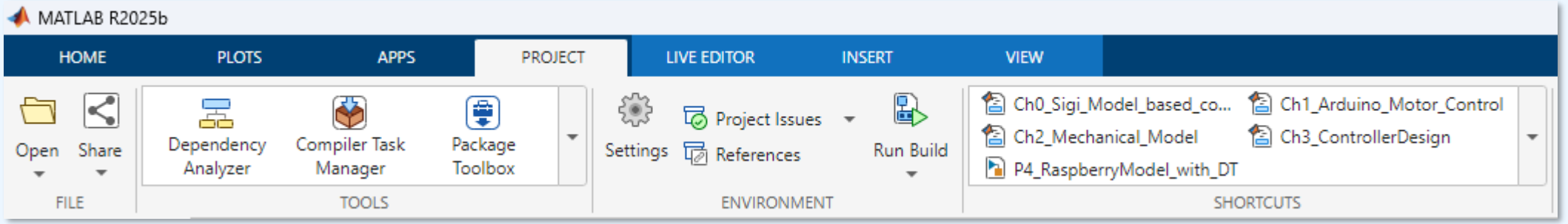


- Integrate PIDs into model for open loop control (used before)
- Deploy controller to the Arduino board (using Support Package)
- Integrate the motor component and the controller into the full system plant
- Use this system plant for development of higher-level controllers

Motor Control Subsystem



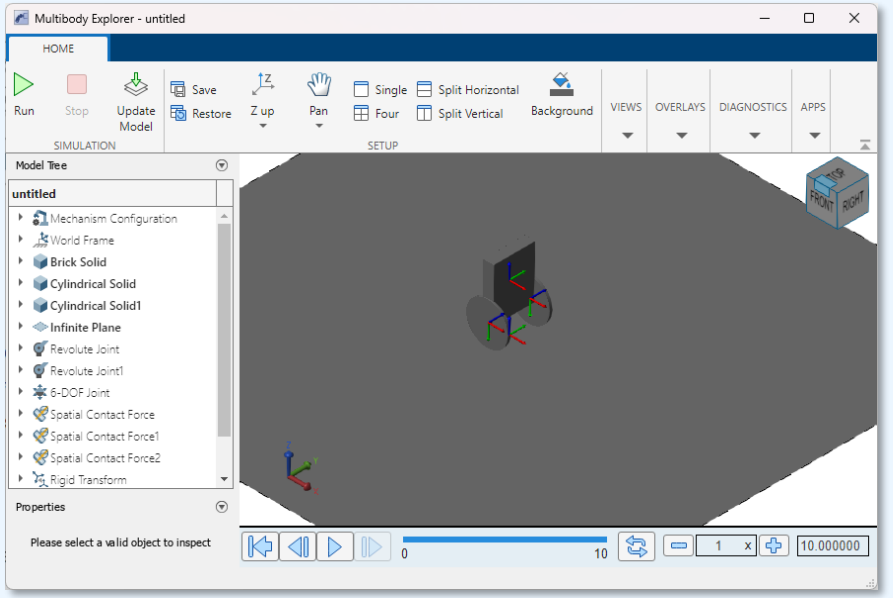
Hands-On Part: Mechanical Model



- Open script Ch2_Mechanical_Model.mlx (Project Shortcut)

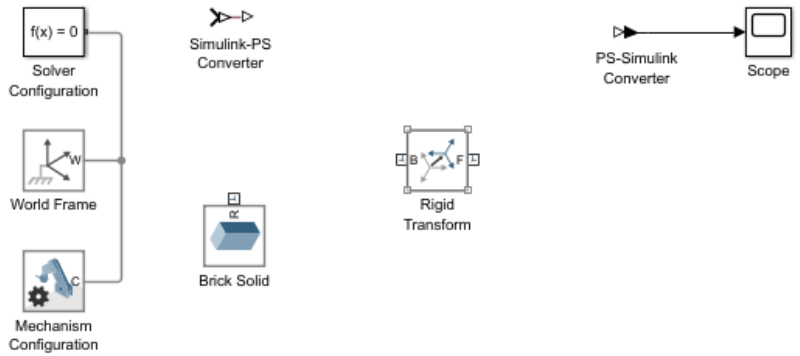
Task:

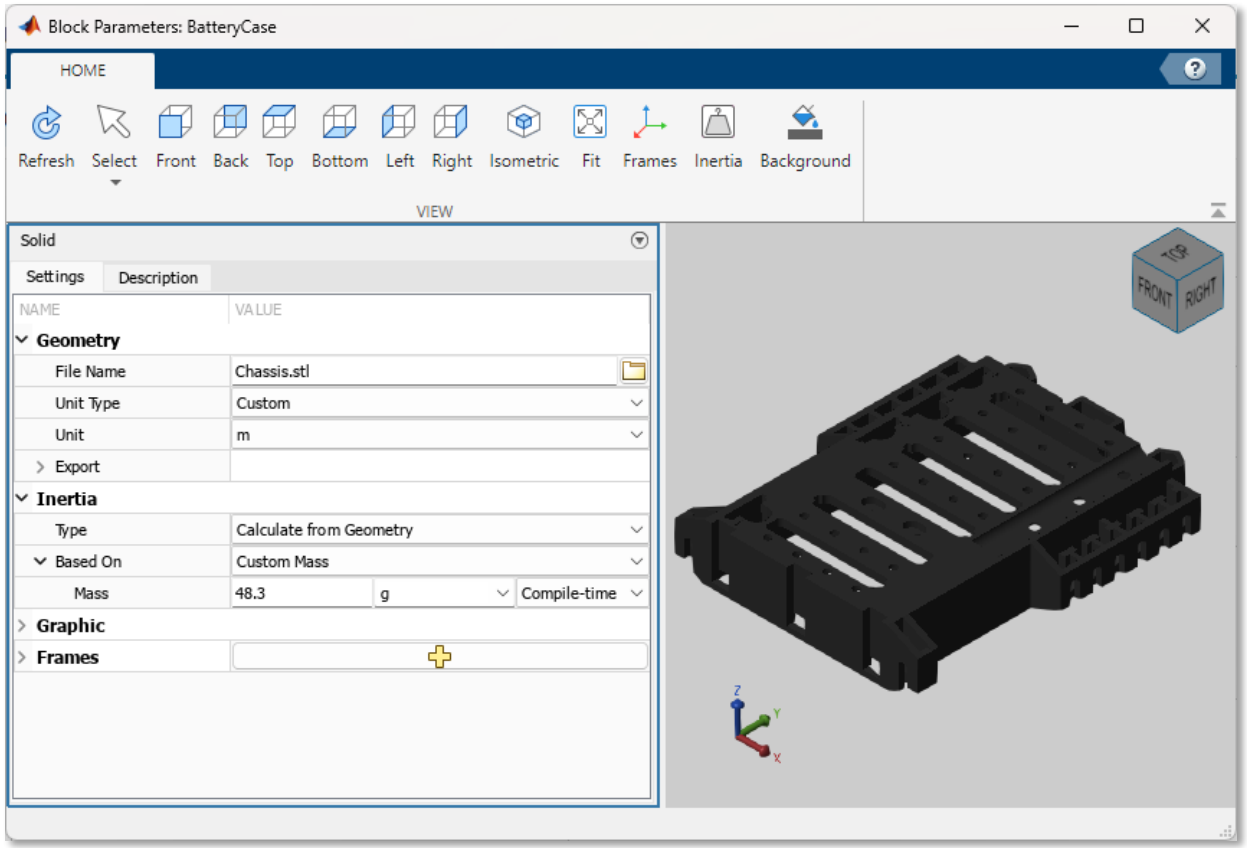
- Add primitives for body, wheels and the floor
- Model interactions with the floor
- Add revolute joints between body and wheels
- Position the robot in the environment



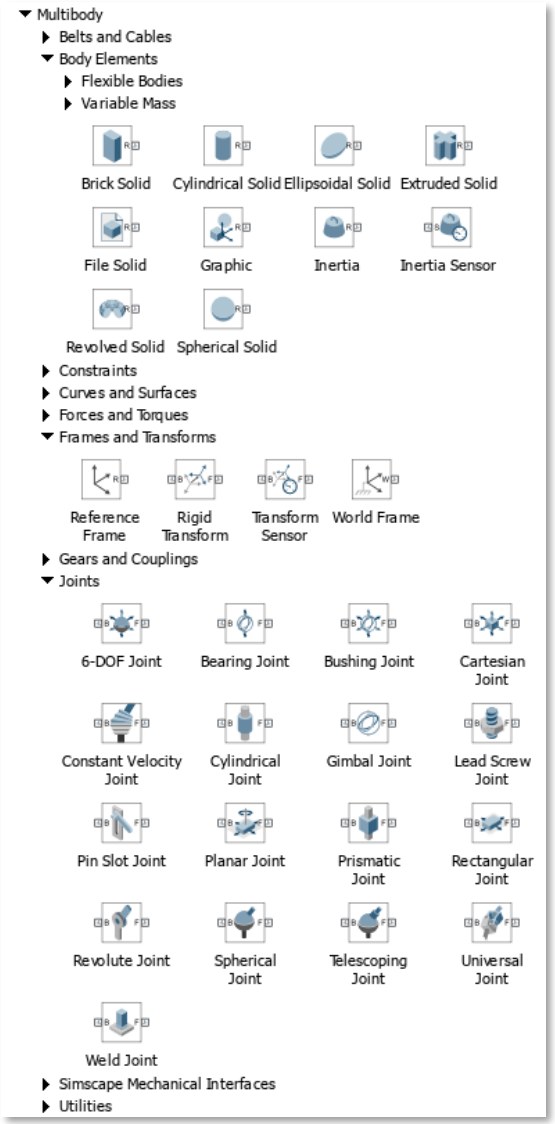
Mechanical modeling with Simscape Multibody

- Add individual components
  - from standard blocks
  - or from CAD files
- Specify component properties
  - such as mass, density etc.

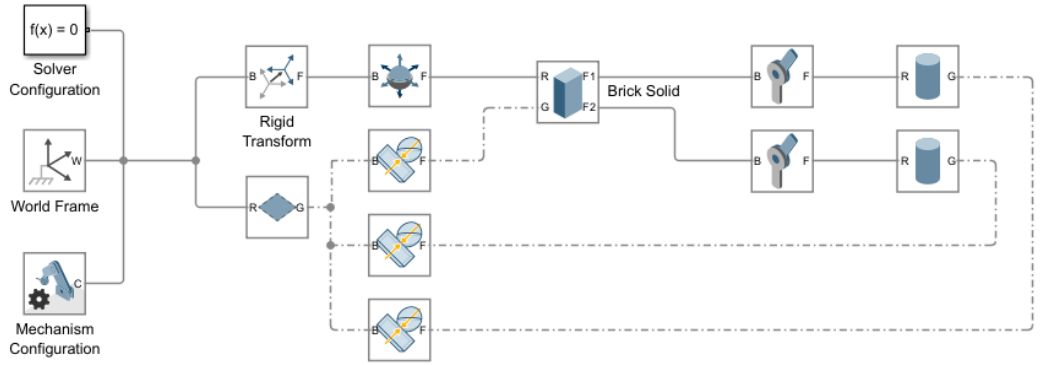




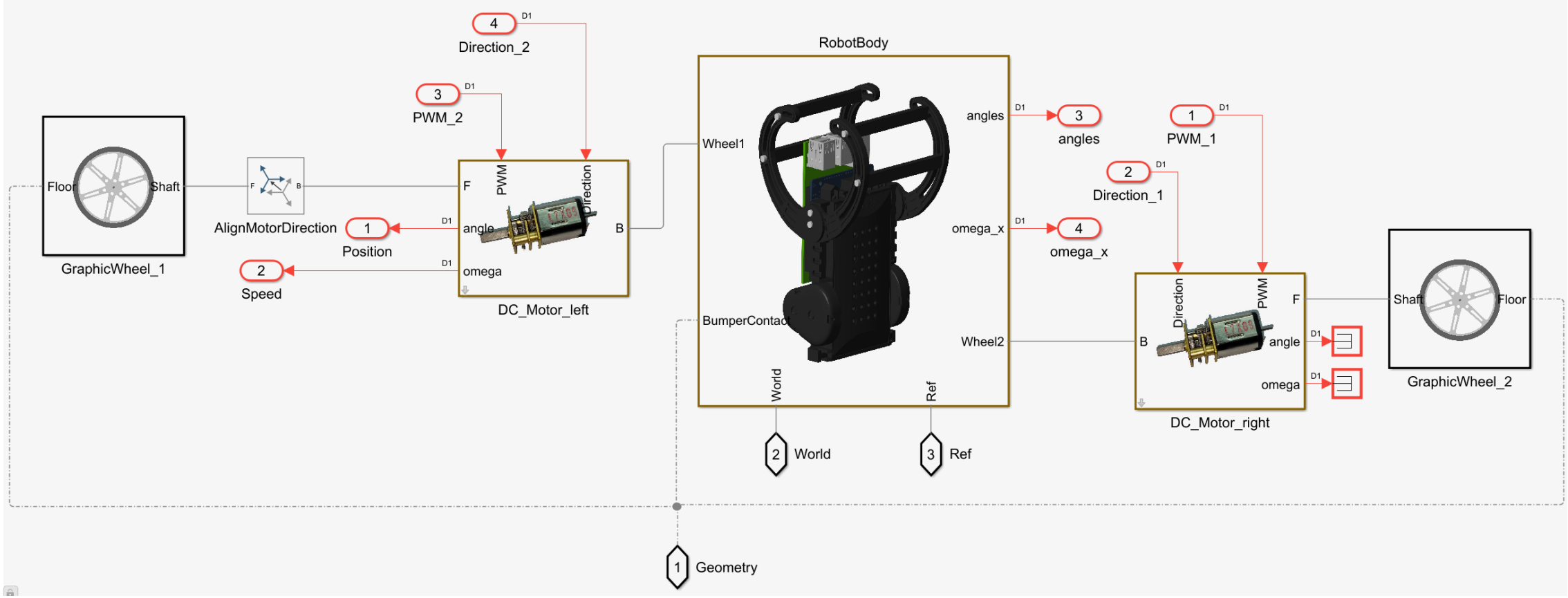
Simscape Multibody: Make a Segway Dummy Model



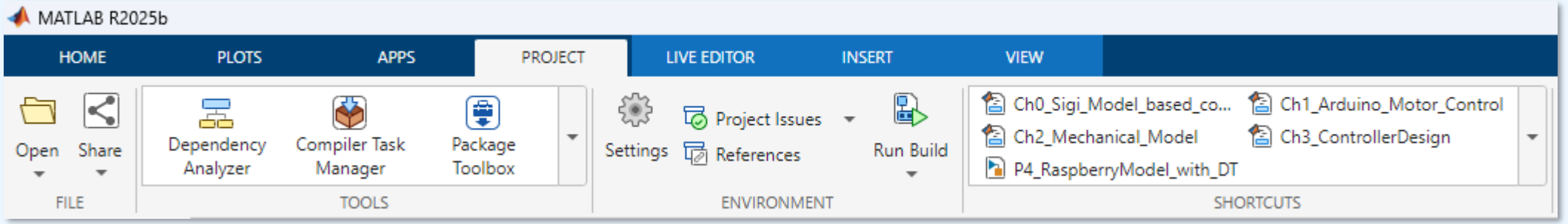
- Connect components to larger assemblies using frames, transforms and joints
- Define contact interaction of components
- Use transforms to measure body properties



Integrate Motors into Mechanical Model



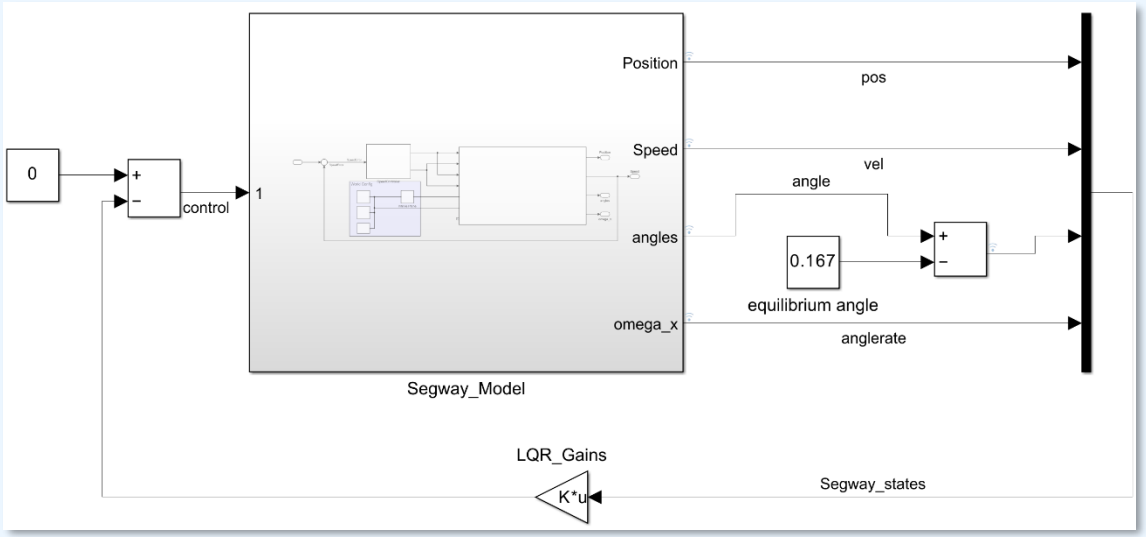
*Hands-On: Balancing Control – LQR implementation



- Open script Ch3_ControllerDesign.mlx (Project Shortcut)

Tasks:

- Linearize Equations of Motion
- Solve Riccati Equation
- Estimate good LQR gains

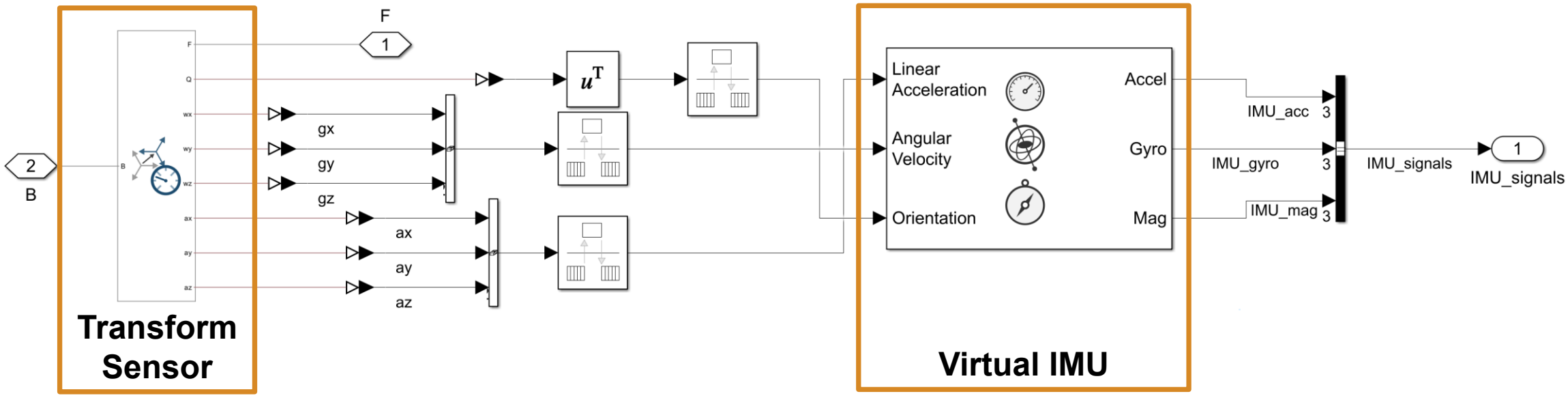


- LQR optimization: https://www.mathworks.com/help/sldo/ug/lqg-controller-tuning.html
- Controls Tech Talks: https://www.mathworks.com/videos/tech-talks/controls.html

What is left to do?

- Sensor Modeling
  - they are typically not ideal -> characterize and add to model
- Estimate robot states using sensor fusion
- Supervisory control needed (Stateflow)
- Validation of controls algorithm
- Deployment of high-level controls to Raspberry Pi

Virtual Sensors ≠ Real sensors

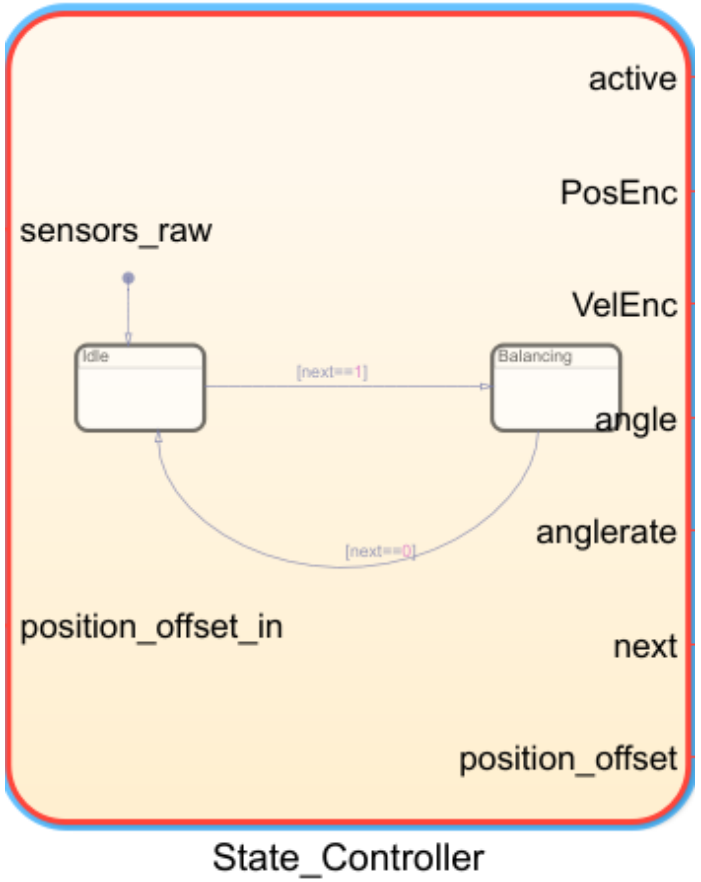


- How to model real sensors (noise, drift, quantization)?
- Characterize sensor properties and use them for simulation
- State estimation from the sensor signals (Sensor Fusion)

- Example sensor characterization: https://www.mathworks.com/help/fusion/ug/inertial-sensor-noise-analysis-using-allan-variance.html
- Sensor Fusion: https://www.mathworks.com/help/fusion/ug/imu-sensor-fusion-with-simulink.html

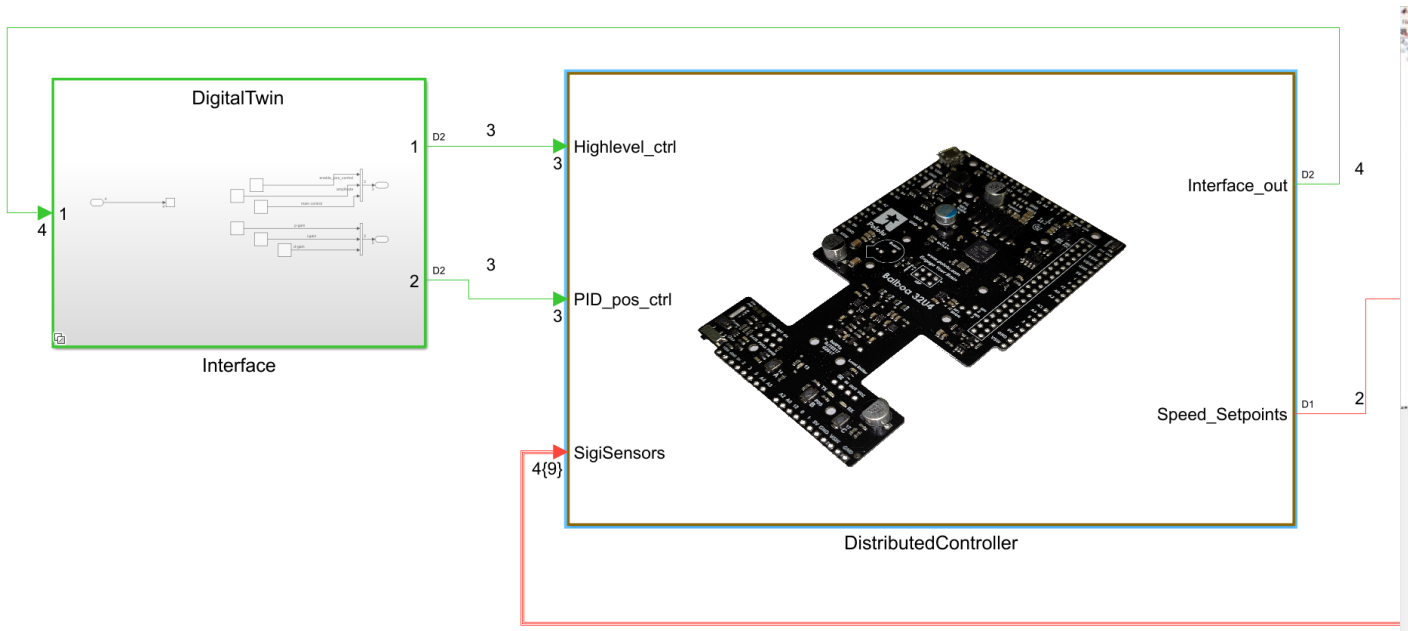
Supervisory Control using Stateflow [1]

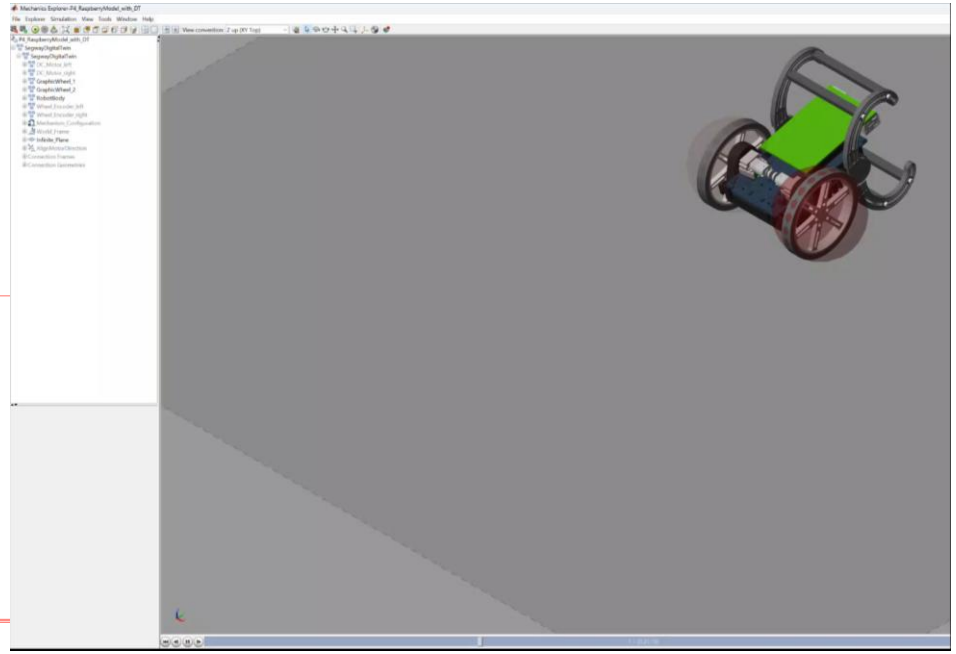
- Model state machine to handle cases like:
  - Balancing (active)
  - Fallen (idle)
  - Calibration
  - Stand-up



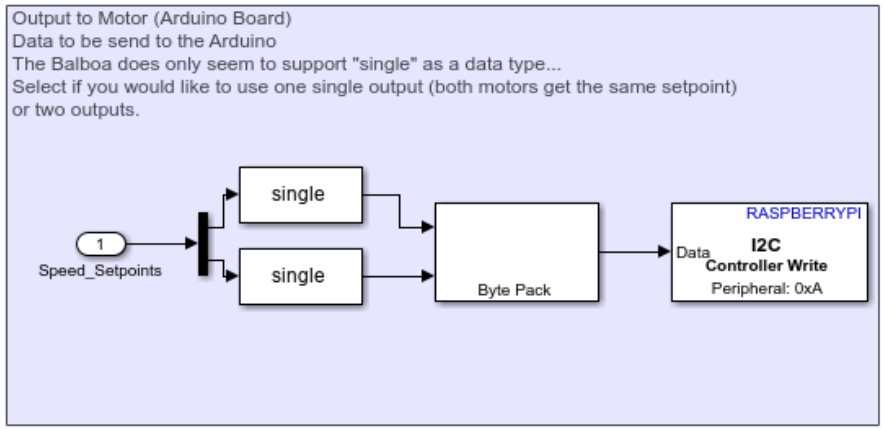
- Stateflow Onramp: https://matlabacademy.mathworks.com/details/stateflow-onramp/stateflow

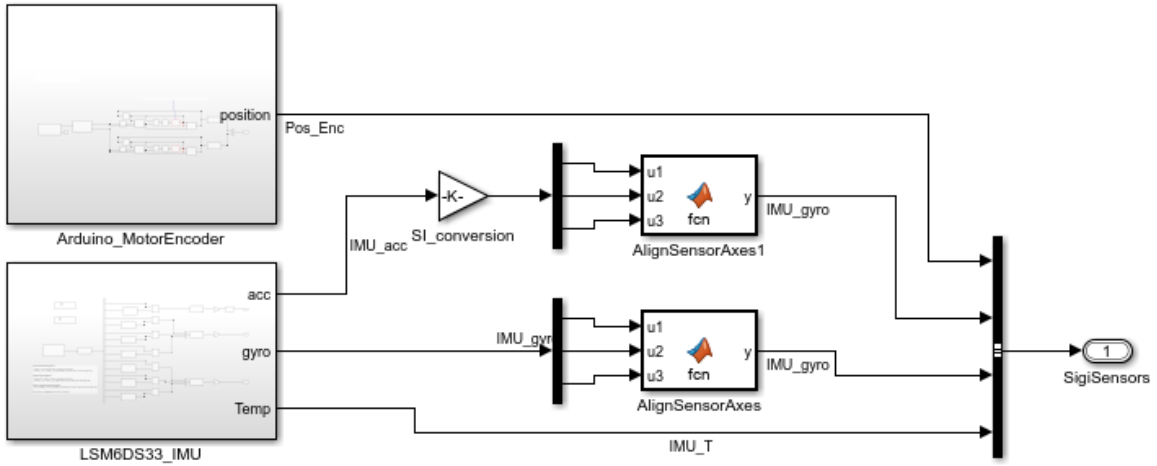
Validation of Model in Simulation





Implement Hardware Connections for Raspberry as Variant Model



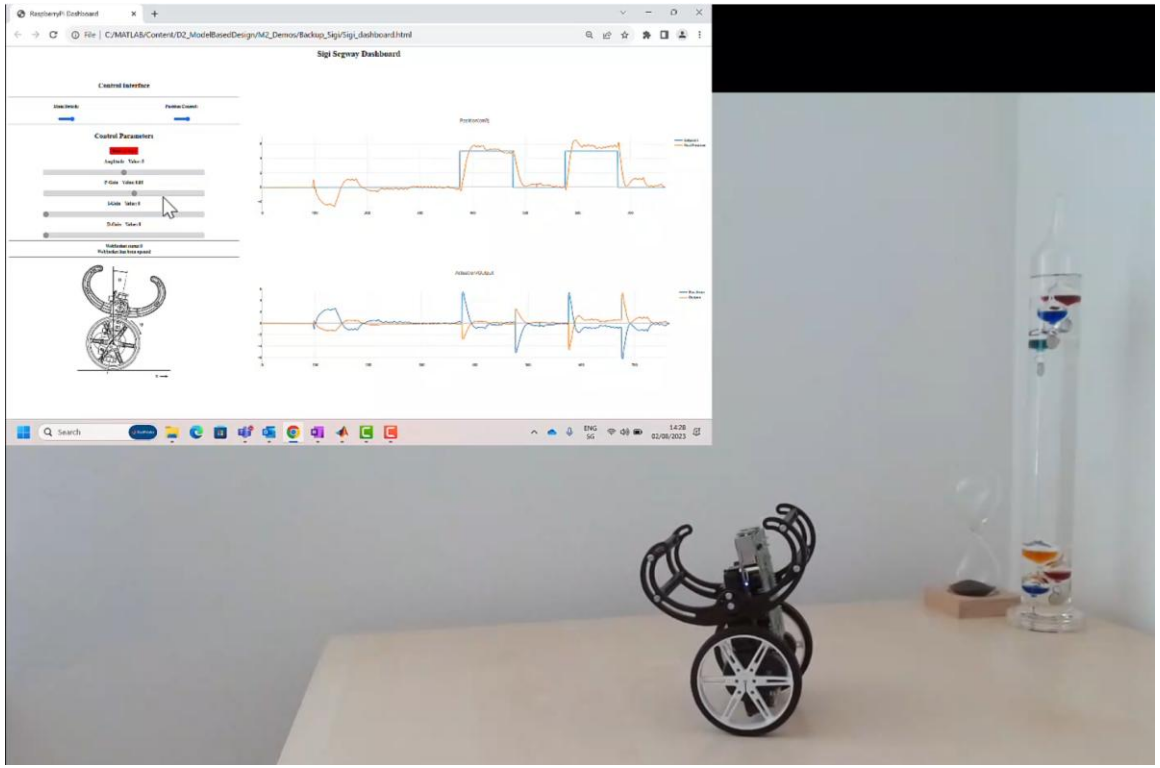


- Configure Simulink to automatically chose variant for code generation
- Embedded Coder translates your algorithm to the target language (C/C++)

- Simulink Support Package for Raspberry Pi: https://www.mathworks.com/matlabcentral/fileexchange/40313-simulink-support-package-for-raspberry-pi-hardware

Remote Control

- Websocket on the Raspberry Pi
  - Interact with the running robot
  - Set setpoints and gains interactively
  - Connection Based on WiFi
  - Platform independent html script



Take Home Message

- Many Apps, examples and features available that can bring you up to speed
  - PID-Tuner App
  - Parameter Optimization App
  - Sensor Modeling and Fusion
- Model physical systems using Simscape and use them for control design.
- Deploy algorithms to hardware using code generation.